SELECTED PROJECTS — ENGINEERING PORTFOLIO

Ashutosh Lal

AI Systems Engineering · Machine Learning Systems · Quantum Computing

`Quantum Technology @ Aalto University · Computer Science Minor · Systems Builder`

WHAT I BUILD

I design and build technically rigorous systems across AI, machine learning, and quantum computing. My work spans retrieval systems, model training pipelines, and scientific simulation—always with a focus on reliability, scalability, and end-to-end execution.

CORE ENGINEERING STRENGTHS

1 I build complete systems end-to-end.

From data ingestion to deployment, I design and implement full technical stacks—not isolated components. In my agentic RAG project, that meant building the entire pipeline: PDF ingestion, chunking, embedding generation, hybrid retrieval (dense + sparse), graph traversal, and multi-agent orchestration on Aalto infrastructure. In my fine-tuning pipeline, it meant owning everything from corpus curation and synthetic data generation to distributed HPC training and evaluation.

2 I optimize for reliability, not just demos.

I design AI systems to fail safely, expose uncertainty, and remain dependable under real-world constraints. My work includes anti-hallucination training pipelines and retrieval systems that prioritize grounded, evidence-based outputs.

3 I learn and execute quickly.

My work spans AI systems engineering, high-performance model training, and quantum computing. I enjoy connecting theory with practical engineering, from retrieval systems and continual fine-tuning pipelines to quantum simulation frameworks.

4 I work across disciplines.

My work spans AI systems engineering, high-performance model training, and quantum computing. I enjoy connecting theory with practical engineering, from retrieval systems and continual fine-tuning pipelines to quantum simulation frameworks.

5 I care about architectural depth.

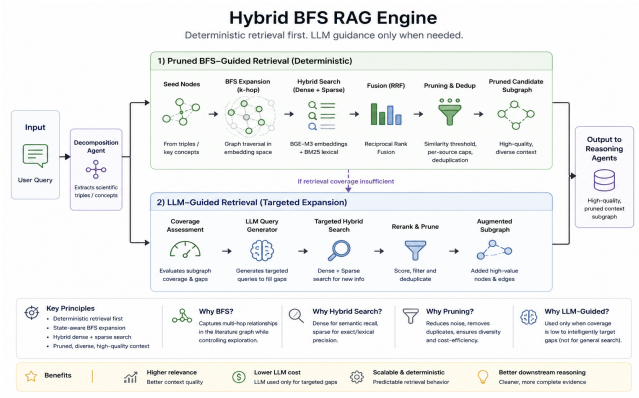
I prioritize modular, scalable system design over quick hacks. My projects are built as composable systems with clear separation between retrieval, reasoning, training, and evaluation layers — designed to evolve, not just function once.

PROJECT 01

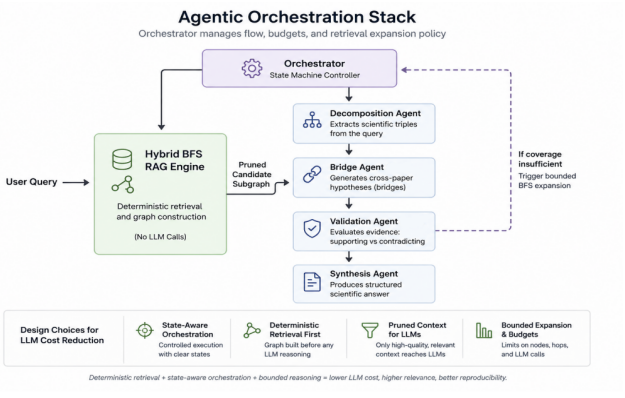
Agentic RAG System for Scientific Literature

Hybrid retrieval + multi-agent reasoning over ~500 quantum ML papers. Shipped on Aalto infrastructure.

LangGraph | LangChain | BGE-M3 + FAISS | BM25 | ChromaDB | Python | Aalto SSH



Hybrid BFS RAG Engine — deterministic retrieval layer



Agentic Orchestration Stack — state-machine reasoning layer

CORPUS
~500 papers, 275k → 45k curated chunks

RETRIEVAL
Dense (BGE-M3) + sparse (BM25), fused via RRF

ORCHESTRATION
LangGraph state machine, max 2–3 LLM calls/query

KEY PRINCIPLE
Deterministic retrieval — no LLM in graph construction

WHAT THIS DEMONSTRATES —

End-to-end ownership. I built every layer: PDF ingestion, embedding pipeline, FAISS index construction, BFS graph traversal, RRF fusion, agent orchestration, synthesis output. The system runs on Aalto infrastructure end-to-end, giving me full ownership over every architectural layer.

Reliability by design. Retrieval is fully deterministic, and no LLM reasoning begins until high-quality evidence has been assembled. All generation is tightly bounded (2–3 calls per query), and the system explicitly signals uncertainty when coverage is insufficient instead of producing speculative outputs. These properties are intentional and not emergent.

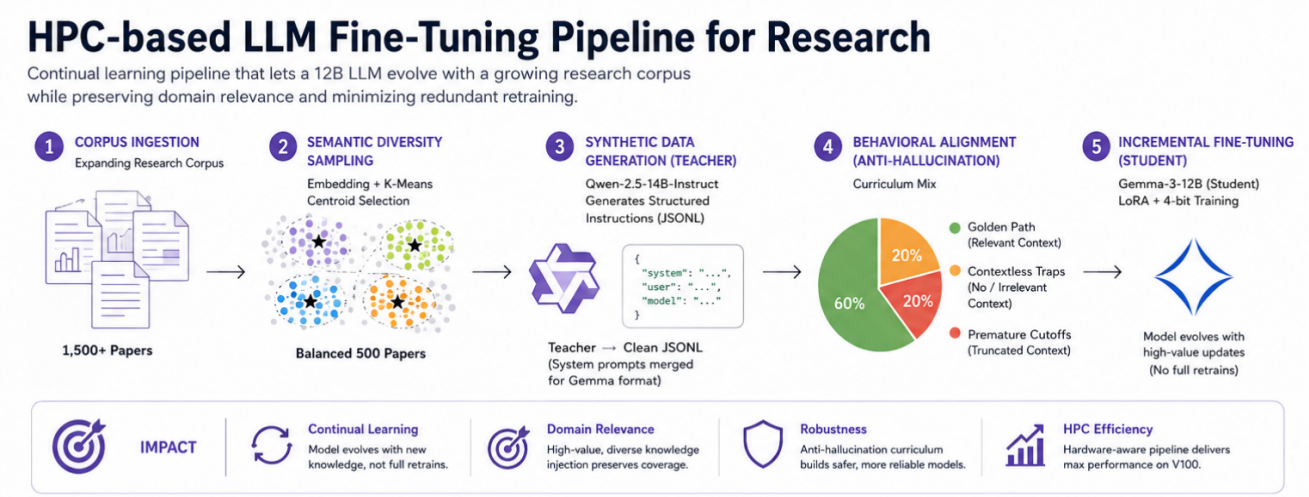
Built under real-world constraints. The system is designed to minimize LLM calls without losing quality. Deterministic components don’t just reduce cost, they improve consistency and make reasoning more reliable.

PROJECT 02

HPC Continual Fine-Tuning Pipeline

Domain-adaptive LLM training with behavioral alignment curriculum. Built during Aaltoes Spark, a two-week MVP sprint program.

LoRA / PEFT | 4-bit NF4 Quantization | SLURM / HPC | Gemma-3-12B | Qwen-14B Teacher | CSC Puhti | Python



`5-stage pipeline: corpus ingestion → semantic diversity sampling → synthetic data generation (teacher) → behavioral alignment → incremental fine-tuning (student)`

STUDENT MODEL
Gemma-3-12B, LoRA adapters, 4-bit NF4

TEACHER MODEL
Qwen-2.5-14B-Instruct → synthetic JSONL

DIVERSITY SAMPLING
K-Means centroid selection, 1,500 → 500 papers

ANTI-HALLUCINATION
60% grounded / 20% traps / 20% cutoffs

WHAT THIS DEMONSTRATES

Engineered reliability. The anti-hallucination curriculum is the core design decision here. 20% of training examples are 'contextless traps' (irrelevant or empty retrieval) where the correct output is a confident refusal. Another 20% are truncated-evidence cases that teach detection of incomplete context. This was not an afterthought. It was the primary design goal.

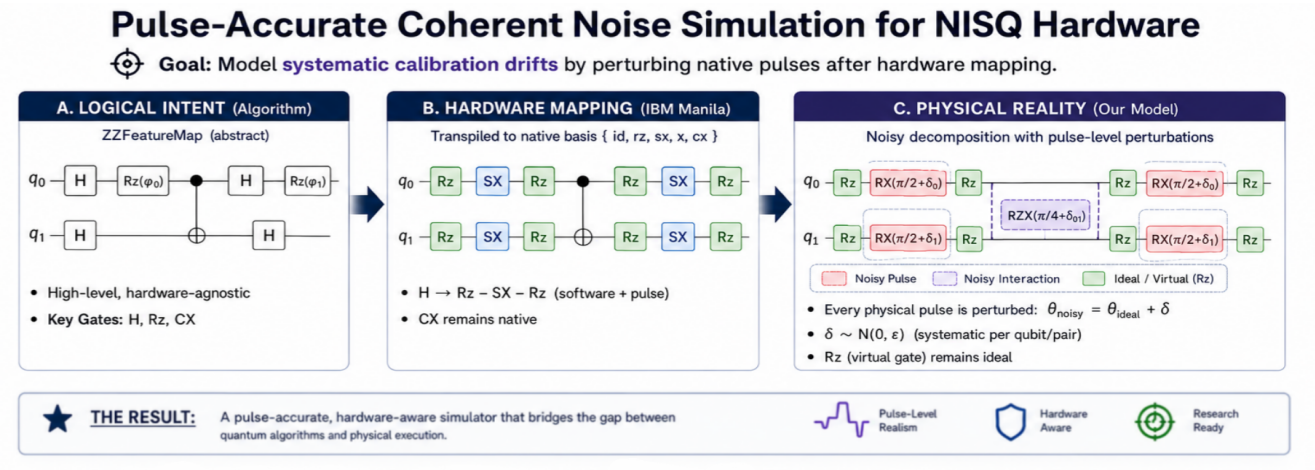
Execution under real constraints. Built on HPC infrastructure with strict memory, compute, and training-budget limits—forcing careful engineering decisions across every stage of the pipeline.

PROJECT 03

Pulse-Accurate Coherent Noise Simulator

Hardware-faithful NISQ simulation modeling systematic calibration drift on IBM superconducting hardware. Built on Qiskit with custom DAG-level circuit rewriting.

Qiskit | DAG Circuit Rewriting | Quantum Kernels | IBM Manila (FakeManilaV2) | Python | NumPy



A → B → C: logical algorithm intent → hardware transpilation (IBM Manila) → pulse-perturbed physical reality

TARGET HARDWARE
IBM Manila — Falcon architecture (FakeManilaV2)

NOISE MODEL
Systematic Gaussian drift per qubit: θ_noisy = θ_ideal + δ

INJECTION POINT
Post-transpilation — after native gate decomposition

BENCHMARK
ZZFeatureMap + SVM quantum kernel on Ad Hoc dataset

KEY DESIGN DECISIONS

Noise injected post-transpilation, not at the logical level. A logical H gate never exists on real hardware — it decomposes into Rz → SX → Rz. Injecting noise before transpilation misses this entirely. This simulator inserts perturbations after hardware mapping, so every noisy operation corresponds to an actual physical pulse.

Systematic drift, not stochastic channels. Most simulators append random error channels after gates. Real calibration drift is different: if a qubit's microwave pulse is miscalibrated, every gate using that pulse inherits the same fixed offset. This simulator assigns drift per qubit, not per gate, capturing coherent error accumulation and interference effects that stochastic models ignore.

DAG-based circuit rewriting for efficiency. Noise injection operates on Qiskit's directed acyclic graph representation — node-level substitution, wire topology preserved, O(n) transformation complexity. Faster and more reliable than rebuilding circuits gate-by-gate.

THE COMMON THREAD

Each project starts from the same question: *what is the actual failure mode, and how do I engineer against it?* In the RAG system, it is hallucination from thin evidence. In the fine-tuning pipeline, it is knowledge collapse and uncontrolled generation. In the noise simulator, it is the systematic, non-random nature of hardware error that stochastic models miss. Technical depth is not the goal — it is the consequence of sophisticated engineering.